

Custom HashMap with String Keys

Hash Tables, Custom Hash Function, Separate Chaining, Rehashing · Mentor-Led Lab

Developed by TalenciaGlobal

Difficulty ★★★ Intermediate

Problem

Design a MyHashMap that maps string keys to integer values, implementing the hash table yourself - no built-in dict, HashMap or unordered_map. Collisions are resolved by separate chaining: each bucket holds a linked list of entries that hash to it.

Support put(key, value) (insert or update), get(key) (the value or -1), and remove(key). Keep the load factor at or below 0.75 by rehashing - doubling the capacity and re-inserting every entry - whenever an insertion pushes it past the threshold.

Learning objectives: understand why hash tables beat direct-address tables, build and tune a string hash function, resolve collisions by chaining, and manage the load factor with automatic rehashing so operations stay average $O(1)$.

Operations & Rules

- PUT key value - insert the pair, or update the value if the key already exists.
- GET key - output the mapped value, or -1 if the key is absent.
- REMOVE key - delete the key and its value if present.
- Hash: a polynomial rolling hash, $\text{hash} = (\text{hash} * 31 + \text{ord}(\text{char})) \bmod \text{capacity}$, applied over the key's characters.
- Load factor: rehash to double the capacity whenever $\text{size} / \text{capacity}$ exceeds 0.75 after an insertion. The map starts at capacity 16.

Input / Output Format

Input: Each line is one operation: PUT key value, GET key, or REMOVE key (read until end of input).

Output: One line per GET (its result), then Size and Capacity of the final table.

Examples

Example 1

Input: put(apple,3); put(banana,5); get(apple); get(mango); put(apple,10); get(apple);
remove(banana); get(banana)

Output:

```
3
-1
10
-1
```

Explanation: apple maps to 3 then is updated to 10; mango is absent (-1); after banana is removed it too returns -1.

Example 2

Input: put(x,1); get(x); remove(x); get(x)

Output:

```
1
-1
```

Explanation: x reads back as 1, then -1 once removed.

Constraints

- $1 \leq \text{key.length} \leq 20$; up to 10,000 operations.
- Implement your own hash table with chaining - no built-in map types.
- Average $O(1)$ per operation; rehash keeps the load factor ≤ 0.75 .

Approach

Hold an array of buckets, each a linked list of (key, value) nodes. The hash maps a key to a bucket index with the polynomial rolling hash modulo the current capacity.

put scans the key's bucket: if the key is present, overwrite its value; otherwise prepend a new node and increment size. get and remove scan the same bucket, returning -1 or unlinking the node respectively.

After an insertion that makes $\text{size} * 4 > \text{capacity} * 3$ (load factor above 0.75), rehash: double the capacity, allocate fresh buckets, and re-insert every existing node (its bucket index changes because the modulus changed).

Quadratic probing and double hashing are open-addressing alternatives to chaining; with chaining the natural tuning knobs are instead the hash multiplier (try primes other than 31) and the load-factor threshold.

Solution

A complete, tested reference solution in each language. All three read the same input and produce identical output.

Python

```
# Lab 1 - Custom HashMap with String Keys (separate chaining)
```

```

import sys

class Node:
    def __init__(self, key, value, nxt=None):
        self.key = key
        self.value = value
        self.next = nxt

class MyHashMap:
    def __init__(self):
        self.capacity = 16
        self.size = 0
        self.buckets = [None] * self.capacity

    def _hash(self, key):
        h = 0
        p = 31
        for ch in key:
            h = (h * p + ord(ch)) % self.capacity
        return h

    def put(self, key, value):
        idx = self._hash(key)
        node = self.buckets[idx]
        while node is not None:
            if node.key == key:
                node.value = value          # update existing
                return
            node = node.next
        self.buckets[idx] = Node(key, value, self.buckets[idx])  # prepend
        self.size += 1
        if self.size * 4 > self.capacity * 3:  # load factor > 0.75
            self._rehash()

    def get(self, key):
        node = self.buckets[self._hash(key)]
        while node is not None:
            if node.key == key:
                return node.value
            node = node.next
        return -1

    def remove(self, key):
        idx = self._hash(key)
        node = self.buckets[idx]
        prev = None
        while node is not None:
            if node.key == key:
                if prev is None:
                    self.buckets[idx] = node.next
                else:
                    prev.next = node.next
                self.size -= 1
                return
            prev = node
            node = node.next

```

```

def _rehash(self):
    old = self.buckets
    self.capacity *= 2
    self.buckets = [None] * self.capacity
    for head in old:
        node = head
        while node is not None:
            idx = self._hash(node.key)
            self.buckets[idx] = Node(node.key, node.value, self.buckets[idx])
            node = node.next

def main():
    m = MyHashMap()
    out = []
    for line in sys.stdin.read().splitlines():
        if not line.strip():
            continue
        p = line.split()
        if p[0] == "PUT":
            m.put(p[1], int(p[2]))
        elif p[0] == "GET":
            out.append(str(m.get(p[1])))
        elif p[0] == "REMOVE":
            m.remove(p[1])
    out.append("Size: " + str(m.size))
    out.append("Capacity: " + str(m.capacity))
    print("\n".join(out))

main()

```

C++

```

// Lab 1 - Custom HashMap with String Keys (separate chaining)
#include <iostream>
#include <string>
#include <sstream>
#include <vector>
using namespace std;
struct Node { string key; int value; Node* next; Node(string k,int v,Node*
n):key(k),value(v),next(n){} };
class MyHashMap {
    int capacity, size;
    vector<Node*> buckets;
    int hashKey(const string& key){
        long long h = 0, p = 31;
        for (char ch : key) h = (h * p + (unsigned char)ch) % capacity;
        return (int)h;
    }
    void rehash(){
        vector<Node*> old = buckets;
        capacity *= 2;
        buckets.assign(capacity, nullptr);
        for (Node* head : old){
            Node* node = head;
            while (node){
                int idx = hashKey(node->key);

```

```

        buckets[idx] = new Node(node->key, node->value, buckets[idx]);
        Node* tmp = node; node = node->next; delete tmp;
    }
}

public:
MyHashMap(): capacity(16), size(0), buckets(16, nullptr) {}
void put(const string& key, int value){
    int idx = hashKey(key);
    for (Node* node = buckets[idx]; node; node = node->next)
        if (node->key == key){ node->value = value; return; }
    buckets[idx] = new Node(key, value, buckets[idx]);
    size++;
    if (size * 4 > capacity * 3) rehash();
}
int get(const string& key){
    for (Node* node = buckets[hashKey(key)]; node; node = node->next)
        if (node->key == key) return node->value;
    return -1;
}
void remove(const string& key){
    int idx = hashKey(key);
    Node* node = buckets[idx]; Node* prev = nullptr;
    while (node){
        if (node->key == key){
            if (prev) prev->next = node->next; else buckets[idx] = node->next;
            delete node; size--; return;
        }
        prev = node; node = node->next;
    }
}
int getSize(){ return size; }
int getCapacity(){ return capacity; }
};

int main(){
    MyHashMap m;
    string line;
    vector<string> out;
    while (getline(cin, line)){
        if (line.empty()) continue;
        istringstream iss(line); string op; iss >> op;
        if (op == "PUT"){ string k; int v; iss >> k >> v; m.put(k, v); }
        else if (op == "GET"){ string k; iss >> k; out.push_back(to_string(m.get(k))); }
        else if (op == "REMOVE"){ string k; iss >> k; m.remove(k); }
    }
    out.push_back("Size: " + to_string(m.getSize()));
    out.push_back("Capacity: " + to_string(m.getCapacity()));
    for (auto& s : out) cout << s << "\n";
    return 0;
}

```

Java

```

// Lab 1 - Custom HashMap with String Keys (separate chaining)
import java.util.*;
public class Main {

```

```

    static class Node { String key; int value; Node next; Node(String k,int v,Node
n){key=k;value=v;next=n;} }
    static class MyHashMap {
        int capacity = 16, size = 0;
        Node[] buckets = new Node[16];
        int hashCode(String key){
            long h = 0, p = 31;
            for (int i = 0; i < key.length(); i++) h = (h * p + key.charAt(i)) % capacity;
            return (int) h;
        }
        void put(String key, int value){
            int idx = hashCode(key);
            for (Node node = buckets[idx]; node != null; node = node.next)
                if (node.key.equals(key)){ node.value = value; return; }
            buckets[idx] = new Node(key, value, buckets[idx]);
            size++;
            if (size * 4 > capacity * 3) rehash();
        }
        int get(String key){
            for (Node node = buckets[hashCode(key)]; node != null; node = node.next)
                if (node.key.equals(key)) return node.value;
            return -1;
        }
        void remove(String key){
            int idx = hashCode(key);
            Node node = buckets[idx], prev = null;
            while (node != null){
                if (node.key.equals(key)){
                    if (prev != null) prev.next = node.next; else buckets[idx] = node.next;
                    size--; return;
                }
                prev = node; node = node.next;
            }
        }
        void rehash(){
            Node[] old = buckets;
            capacity *= 2;
            buckets = new Node[capacity];
            for (Node head : old)
                for (Node node = head; node != null; node = node.next){
                    int idx = hashCode(node.key);
                    buckets[idx] = new Node(node.key, node.value, buckets[idx]);
                }
        }
    }
}

public static void main(String[] args){
    Scanner sc = new Scanner(System.in);
    MyHashMap m = new MyHashMap();
    StringBuilder sb = new StringBuilder();
    while (sc.hasNextLine()){
        String line = sc.nextLine().trim();
        if (line.isEmpty()) continue;
        String[] p = line.split("\\s+");
        if (p[0].equals("PUT")) m.put(p[1], Integer.parseInt(p[2]));
        else if (p[0].equals("GET")) sb.append(m.get(p[1])).append("\n");
        else if (p[0].equals("REMOVE")) m.remove(p[1]);
    }
    sb.append("Size: ").append(m.size).append("\n");
}

```

```
        sb.append("Capacity: ").append(m.capacity).append("\n");  
        System.out.print(sb);  
    }  
}
```

Sample Run

Sample Input

```
PUT apple 3  
PUT banana 5  
GET apple  
GET mango  
PUT apple 10  
GET apple  
REMOVE banana  
GET banana  
PUT cherry 1  
PUT date 2  
PUT fig 4  
PUT grape 6  
PUT kiwi 7  
PUT lemon 8  
PUT mango 9  
PUT melon 11  
PUT nut 12  
PUT olive 13  
PUT pear 14  
PUT plum 15  
GET melon  
GET plum
```

Sample Output

```
3  
-1  
10  
-1  
11  
15  
Size: 13  
Capacity: 32
```

Follow-Up & Discussion

- If the hash function always returns 0, every key lands in one bucket: the chain becomes a linked list and put/get/remove degrade to $O(n)$ - the hash table collapses into linear search.
- To switch to open addressing (linear probing), store entries directly in the table and on a collision probe $h(k)+1$, $h(k)+2$, ... for the next free slot, using tombstones for deletion (this is exactly the next lab).

- For a high-concurrency Fintech transaction log, make it thread-safe with fine-grained locking (a lock per bucket / lock striping) or a lock-free design, so independent buckets can be updated in parallel.
- Experiment: swap the multiplier 31 for a poor choice (say 1, which sums character codes) and watch collisions and probe-chain lengths balloon - good hash functions spread keys uniformly.